

Jordan Rivers

City, ST • jordan.rivers@example.com • linkedin.com/in/jordanrivers

Summary

- Senior software engineer with 8+ years building **scalable backend services** and distributed systems in production
- Specialize in **cloud-native microservices** on AWS and Kubernetes, with a focus on reliability, performance, and cost
- Partner closely with product and SRE teams to ship **well-tested APIs, automation, and observability** at scale

Education & Skills

Education: B.S. in Computer Science, Lakemont University, 2015

Programming Languages: Python, Java, Go, JavaScript, TypeScript, SQL, Bash

Skills: AWS, Docker, Kubernetes, Terraform, PostgreSQL, Redis, gRPC, REST APIs, microservices, distributed systems, event-driven architecture, CI/CD, Git, observability

Experience

Northwind Systems – Senior Software Engineer

2016 – Present | City, ST

Payments platform microservices migration

- Led migration of a **monolithic payments service** into independently deployable microservices, cutting deploy time from hours to minutes and enabling isolated, low-risk releases
- Designed **idempotent transaction workflows** and retry semantics with exactly-once settlement guarantees that reduced failed-payment incidents by **40%** during peak traffic
- Introduced **contract tests and canary releases** so teams could ship independently without cross-team coordination, catching breaking changes before they reached production

Real-time notifications and event pipeline

- Built an **event-driven notifications pipeline** processing 50M+ daily events with at-least-once delivery guarantees and idempotent, replayable consumers
- Implemented **backpressure and dead-letter queues** to keep the system stable during downstream outages and traffic spikes, with automated replay of failed messages
- Cut end-to-end notification latency from minutes to **under two seconds** at the 99th percentile by batching, connection pooling, and tuning consumer concurrency

Public REST and gRPC API gateway

- Designed and shipped a **public API gateway** exposing REST and gRPC endpoints to thousands of third-party developers, with schema-driven routing and validation
- Added **authentication, rate limiting, and request validation** as reusable middleware shared across services, standardizing security controls platform-wide
- Authored versioned API docs and **client SDKs** in multiple languages that reduced partner integration time from weeks to days and cut integration support tickets

Observability and incident response tooling

- Rolled out **structured logging, metrics, and distributed tracing** across 30+ services for end-to-end visibility into cross-service request flows
- Built **on-call dashboards and automated runbooks** that cut mean time to resolution by roughly **35%** and reduced repetitive pages for the on-call rotation
- Established **service-level objectives** and alerting policies adopted as the team standard for every new service, tying alerts to real user-facing impact

Customer onboarding automation service

- Automated **customer onboarding workflows** with a self-service portal, replacing manual, error-prone provisioning with an auditable, repeatable pipeline
- Cut new-customer setup time from days to under an hour and removed a recurring class of provisioning errors through validation and automated rollbacks
- Partnered with support and product teams to define **self-serve usage tiers and limits** for new tenants, reducing back-and-forth during account setup